Aula 10 - Diretivas de Compilação

Vetorização com o construtor `simd`

Prof. Rogério Aparecido Gonçalves[1]
rogerioag@utfpr.edu.br

[1]Universidade Tecnológica Federal do Paraná (UTFPR)
Departamento de Computação (DACOM)
Campo Mourão - Paraná - Brasil

Programa de Pós Graduação em Ciência da Computação
Mestrado em Ciência da Computação
PPGCC18 - Computação Paralela e Distribuída

Este trabalho está licenciado com uma Licença Creative Commons Teste - Atribuição-NãoComercial-SemDerivações 4.0 Internacional.

Agenda i

1. Introdução
2. Suporte à Vetorização
3. Suporte à Vetorização no `OpenMP`: `construtor simd`
4. Cláusulas do construtor `SIMD`
5. Construtor `#pragma omp declare simd`
6. Uso combinado do construtor `SIMD`
7. Atividades sobre *simd*

Agenda ii

8. Dúvidas

9. Referências

Introdução

Objetivos

- Explorar o Construtor para Vetorização: **`SIMD`**.
- Testar os efeitos do construtor **`simd`** e as cláusulas admitidas[1], bem como a aplicação da cláusula **`simd`** nos construtores já vistos, como **`for simd`** e *taskloop simd*.

1

Suporte à Vetorização

O que é SIMD i

- SIMD é uma das categorias da Taxonomia de Flynn [M. J. Flynn and Rudd (1996)](M. Flynn 2011), SIMD (Single Instruction Multiple Data) – Fluxo único de instruções em múltiplos conjuntos de dados.
- A capacidade de Vetorização está relacionada com um conjunto de instruções que são capazes de executar operações sobre um conjunto de dados/operandos.
- Instruções escalares vs. Instruções vetoriais.
- Para verificar se o processador da sua máquina tem extensões do conjuntos de instruções que possibilitam o suporte a vetorização, o comando `cat /proc/cpuinfo` pode ser utilizado.

O que é SIMD ii

Terminal

```
rag@chamonix~$ cat /proc/cpuinfo | grep flags | tail -n 1
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca
    cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe
    syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts
    rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni
    pclmulqdq dtes64 monitor ds_cpl vmx est tm2 ssse3 sdbg fma cx16
    xtpr pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt
    tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm
    cpuid_fault epb invpcid_single pti ssbd ibrs ibpb stibp
    tpr_shadow vnmi flexpriority ept vpid ept_ad fsgsbase tsc_adjust
    bmi1 avx2 smep bmi2 erms invpcid xsaveopt dtherm ida arat pln
    pts md_clear flush_l1d
rag@chamonix~$
```

- Os *flags* `sse`, `sse2`, `fma`, `sse4_1`, `sse4_2`, `avx` e `avx2` indicam o suporte do processador a instruções de vetorização.

Intel SIMD Extensions i

- Novas instruções, novos registradores.
- Suporte introduzido em fases ou grupos de funcionalidades.
  - **SSE - SSE** (1999 - 2006)
    - Operações de 128 bits.
  - **AVX**, **FMA**, **AVX2** e **AVX-512** (2008 - 2015)
    - Operações de 256 e 512 bits.
    - Chips AVX-512 disponíveis em **Intel Xeon Phi**.

Intel SIMD Extensions ii

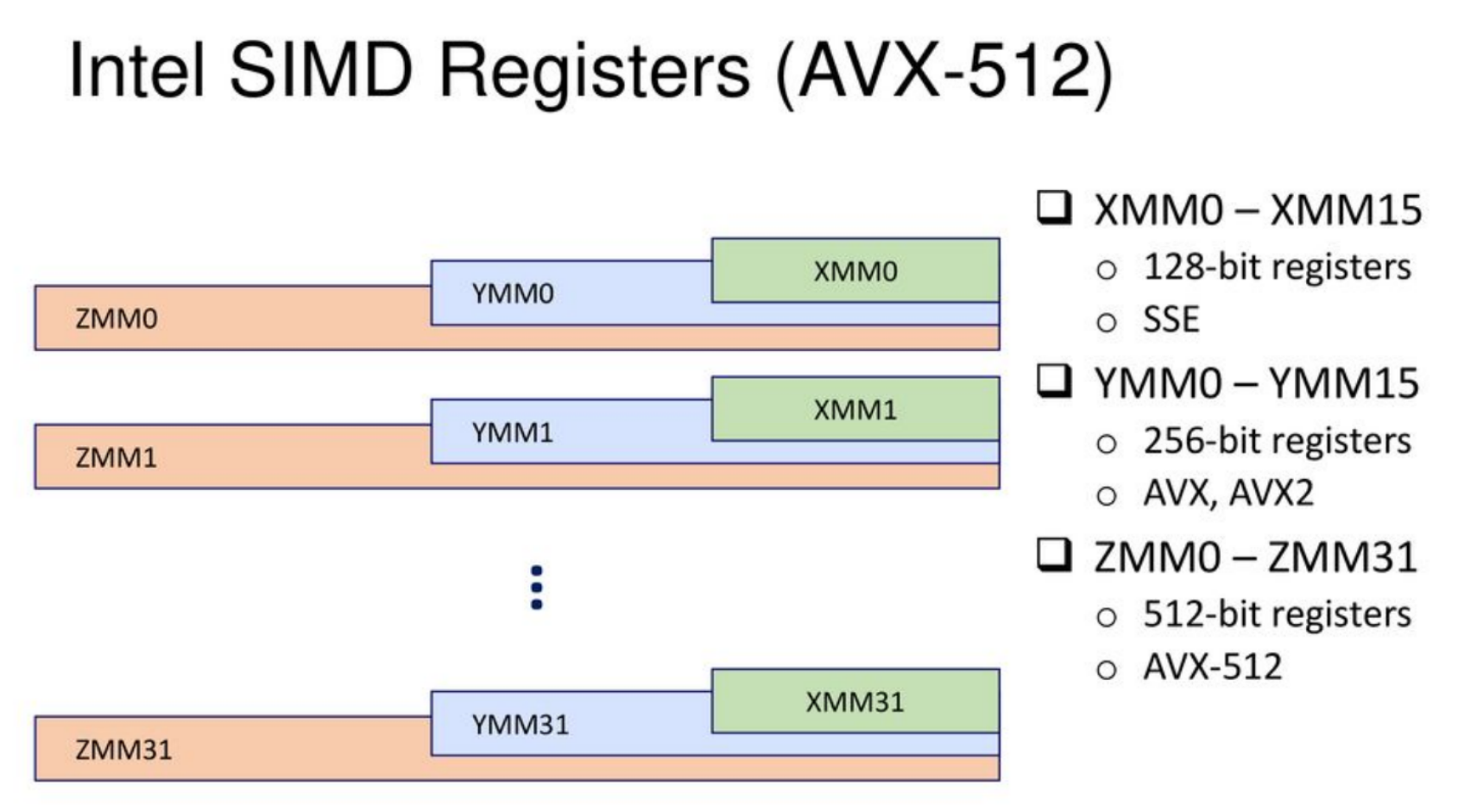


Figura 1: Registradores SIMD

Intel SIMD Extensions iii

- No GCC é possível habilitar a utilização automática de vetorização com os *flags*:
- `-O3 -mavx`: 128 bits
- `-mavx2`: 256 bits
- `-mamx-avx512`: 512 bits.
- Para testar e visualizar o código gerado, o https://gcc.godbolt.org pode ser utilizado.
- Considere o exemplo de multiplicação de dois `arrays` de inteiros:

```
int a[256], b[256], c[256];
int main(){
    for (int i=0; i<256; i++){
        c[i] = a[i] * b[i];
    }
```

Intel SIMD Extensions iv

```
    return 0;
}
```

- Outras opções podem ser [testadas](https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html:

Flag	Descrição
`-mmmx`	Support MMX built-in functions.
`-msse`	Support MMX and SSE built-in functions and code generation.
`-msse2`	Support MMX, SSE and SSE2 built-in functions and code generation.
`-msse3`	Support MMX, SSE, SSE2 and SSE3 built-in functions and code generation.

Intel SIMD Extensions v

`-mssse3`	Support MMX, SSE, SSE2, SSE3 and SSSE3 built-in functions and code generation.
`-msse4.1`	Support MMX, SSE, SSE2, SSE3, SSSE3 and SSE4.1 built-in functions and code generation.
`-msse4.2`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1 and SSE4.2 built-in functions and code generation.
`-msse4`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1 and SSE4.2 built-in functions and code generation.
`-msse4a`	Support MMX, SSE, SSE2, SSE3 and SSE4A built-in functions and code generation.
`-mavx`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2 and AVX built-in functions and code generation.
`-mavx2`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX and AVX2 built-in functions and code generation.

Intel SIMD Extensions vi

`-mavx512f`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2 and AVX512F built-in functions and code generation.
`-mavx512cd`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512CD built-in functions and code generation.
`-mavx512vl`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512VL built-in functions and code generation.
`-mavx512bw`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512BW built-in functions and code generation.
`-mavx512dq`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512DQ built-in functions and code generation.

Intel SIMD Extensions vii

`-mavx512ifma` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512IFMA built-in functions and code generation.

`-mavx512vbmi` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512VBMI built-in functions and code generation.

`-mavx512vpopcntdq` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512VPOPCNTDQ built-in functions and code generation.

`-mavx512vp2intersect` Support AVX512VP2INTERSECT built-in functions and code generation.

`-mavx512vnni` Support AVX512VNNI built-in functions and code generation.

`-mavx512vbmi2` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512VBMI2 built-in functions

Intel SIMD Extensions viii

`-mavx512bf16` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512BF16 built-in functions and code generation.

`-mavx512fp16` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512-FP16 built-in functions and code generation.

`-mavx512bitalg` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX512F and AVX512BITALG built-in functions and code generation.

`-mavx512bmm` Support AVX512BMM built-in functions and code generation.

`-mavx512vaes` Support AVX512VAES built-in functions and code generation.

`-mavx512vnni` Support AVX512VNNI built-in functions and code generation.

Intel SIMD Extensions ix

`-mavx512ifma` Support AVX512IFMA built-in functions and code generation.

`-mavx5124fmaps` Support AVX5124FMAPS built-in functions and code generation.

`-mavx5124vnniw` Support AVX5124VNNIW built-in functions and code generation.

`-mavx512vvl` Support AVX512VVL built-in functions and code generation.

`-mavx512vbw` Support AVX512BW built-in functions and code generation.

`-mavx512vdq` Support AVX512DQ built-in functions and code generation.

`-mavxvnni` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, and AVXVNNI built-in functions and code generation.

Intel SIMD Extensions x

`-mavxifma` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, and AVXIFMA built-in functions and code generation.

`-mavxvnniint8` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2 and AVXVNNIINT8 built-in functions and code generation.

`-mavxneconvert` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, and AVXNECONVERT build-in functions and code generation.

`-mavxvnniint16` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2 and AVXVNNIINT16 built-in functions and code generation.

`-mavx10.1` Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, and AVX10.1 built-in functions and code generation.

Intel SIMD Extensions xi

`-mavx10.2`	Support MMX, SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX10.1 and AVX10.2 built-in functions and code generation.

Intel SIMD Extensions xii

: Flags de vetorização

- Utilizando o GCC 8.3 em um processador Intel X86_64, teremos o código *assembly* gerado:

```
main:
    xor     eax, eax
.L2:
    vmovdqu xmm2, XMMWORD PTR a[rax]
    vmovdqu xmm3, XMMWORD PTR b[rax]
    add     rax, 32
    vinserti128     ymm1, ymm2, XMMWORD PTR a[rax-16], 0x1
    vinserti128     ymm0, ymm3, XMMWORD PTR b[rax-16], 0x1
    vpmulld ymm0, ymm0, ymm1
    vmovups XMMWORD PTR c[rax-32], xmm0
    vextracti128    XMMWORD PTR c[rax-16], ymm0, 0x1
    cmp     rax, 1024
    jne     .L2
    vzeroupper
```

```
    xor     eax, eax
    ret
```

- Utilizando o GCC 10.2 em um processador Intel X86_64, teremos o código *assembly* gerado:

```
main:
    xor     eax, eax
.L2:
    vmovdqa ymm1, YMMWORD PTR a[rax]
    add     rax, 32
    vpmulld ymm0, ymm1, YMMWORD PTR b[rax-32]
    vmovdqa YMMWORD PTR c[rax-32], ymm0
    cmp     rax, 1024
    jne     .L2
    xor     eax, eax
    vzeroupper
    ret
```

Intel SIMD Extensions xiv

- Na máquina **Intel(R) Core(TM) i7-4700HQ CPU @ 2.40GHz, quad-core com HT, arch: Haswell**, utilizando GCC 10.1.

```
    .file   "example-simd-gcc.c"
    .text
    .section    .text.startup,"ax",@progbits
    .p2align 4
    .globl  main
    .type   main, @function
main:
    xorl    %eax, %eax
    leaq    c(%rip), %rsi
    leaq    a(%rip), %rcx
    leaq    b(%rip), %rdx
    .p2align 4,,10
    .p2align 3
.L2:
    vmovdqa (%rcx,%rax), %ymm1
```

Intel SIMD Extensions xv

```
    vpmulld (%rdx,%rax), %ymm1, %ymm0
    vmovdqa %ymm0, (%rsi,%rax)
    addq    $32, %rax
    cmpq    $1024, %rax
    jne .L2
    xorl    %eax, %eax
    vzeroupper
    ret
    .size   main, .-main
...
    .ident  "GCC: (GNU) 10.1.0"
    .section    .note.GNU-stack,"",@progbits
```

Intel SIMD Intrinsics i

- *Compiler Intrinsics* ou *builtins* são funções otimizadas embutidas no compilador que permitem ao invés de colocarmos um código *assembly inline* para chamar as instruções `AVX` utilizando funções "mais alto" nível.
- Compilador gerencia todos os recursos, como registradores.
- **Link:** *Intel SIMD Intrinsics*

Intel SIMD Intrinsics ii

Multiplicação dos arrays de 256 elementos inteiros

Função: __m256d _mm256_mul_pd (__m256d a, __m256d b)

Necessário: `#include <immintrin.h>`

Instruções: `vmulpd ymm, ymm, ymm`

CPUID Flags: `AVX`

Description: *Multiply packed double-precision (64-bit) floating-point elements in a and b, and store the results in dst.*

Intel SIMD Intrinsics iii

- O exemplo de multiplicação de *arrays* utilizando funções *builtins*:

```
#include <immintrin.h>

// int a[256], b[256], c[256];
__m256d a,b,c;

int main(){
  //for (int i=0; i<256; i++){
  //      c[i] = a[i] * b[i];
  //}
  c = _mm256_mul_pd(a,b);

  return 0;
}
```

Intel SIMD Intrinsics iv

- **Artigo:** An Introduction to GCC Compiler Intrinsics in Vector Processing
- **Documentação do GCC**: *6.52 Using Vector Instructions through Built-in Functions*

Suporte à Vetorização no OpenMP: `construtor simd`

Vetorização de Código: construtor *simd* i

- Diretiva: `#pragma omp simd`

Definição

- O construtor **`simd`** pode ser aplicado a um laço diretamente indicando que múltiplas iterações do laço podem ser executadas concorrentemente usando instruções **`SIMD`**.

No documento de especificação *OpenMP 4.5*: *2.8.1 simd Construct*

No documento de especificação *OpenMP 5.0*: *2.8.1 simd Construct*

Sintaxe

```
#pragma omp simd [clause[[,] clause] ...]new-line
    for-loops
```

Vetorização de Código: construtor *simd* ii

- **Exemplo:** Utilização do construtor **`simd`** em um laço que faz a multiplicação de dois *arrays*.

```
int main(int argc, char **argv) {
  int i;
  double res;
  init_array();

  #pragma omp simd
  for (i = 0; i < N; i++) {
    h_c[i] += h_a[i] * h_b[i];
  }

  return 0;
}
```

Vetorização de Código: construtor *simd* iii

- O construtor **simd** não apresenta um formato específico.
- Código do corpo do laço terá instruções **SIMD**.

```
.L6:
  movl   -20(%rbp), %eax
  cltq
  movsd h_c(,%rax,8), %xmm1
  movl   -20(%rbp), %eax
  cltq
  movsd h_a(,%rax,8), %xmm2
  movl   -20(%rbp), %eax
  cltq
  movsd h_b(,%rax,8), %xmm0
  mulsd %xmm2, %xmm0
  addsd %xmm0, %xmm1
  movq   %xmm1, %rax
  movl   -20(%rbp), %edx
  movslq  %edx, %rdx
  movq   %rax, h_c(,%rdx,8)
  addl   $1, -20(%rbp)
```

Cláusulas do construtor `SIMD`

Cláusulas do construtor SIMD i

Cláusulas Admitidas OpenMP 4.0

```
safelen(length)
linear(list[:linear-step])
aligned(list[:alignment])
private(list)
lastprivate(list)
reduction(reduction-identifier:list)
collapse(n)
```

Cláusulas Adicionadas OpenMP 4.5

```
simdlen(length)
```

Cláusulas do construtor `SIMD` ii

Cláusulas Adicionadas OpenMP 5.0 e 5.2

```
if([simd :] scalar-expression)
nontemporal(list)
reduction([ reduction-modifier,]reduction-identifier : list)
order(concurrent)
```

Cláusulas Adicionadas OpenMP 6.0

```
induction
```

Cláusulas do construtor `SIMD` iii

1. **`if([simd :] scalar-expression)`**: Cláusula Condicional para `SIMD`.

2. **`safelen(length)`**: Especifica que duas iterações simultâneas em um bloco SIMD não podem ter uma distância no espaço de iteração lógica maior ou igual ao valor fornecido na cláusula. O parâmetro da cláusula safelen deve ser uma expressão de número inteiro positivo constante.

3. **`simdlen(length)`**: Especifica o número preferencial de iterações a serem executadas simultaneamente, a menos que uma cláusula **`if`** esteja presente e seja avaliada como falsa, caso em que o número preferencial de iterações a serem executadas simultaneamente é um. O parâmetro da cláusula simdlen deve ser uma expressão de número inteiro positivo constante.

Cláusulas do construtor `SIMD` iv

4. **`linear(list[ : linear-step])`**: Declara um ou mais itens da lista como privados e tendo um relacionamento linear com o espaço de iteração de um *loop* associado ao construtor na qual a cláusula aparece.

5. **`aligned(list[ : alignment])`**: Declara que o objeto para o qual cada item da lista aponta está alinhado ao número de bytes expresso no parâmetro opcional da cláusula.

6. **`nontemporal(list)`**: Especifica que os acessos para o local de armazenamento para o qual cada item da lista referencia tem baixa localidade temporal através das iterações nas quais esses locais são acessados.

Cláusulas do construtor `SIMD` v

7. **`private(list)`**: deixa as variáveis da lista marcadas como privativas de cada *thread*.

8. **`lastprivate([ lastprivate-modifier:] list)`**: Declara um ou mais itens de uma lista para ser privados para uma tarefa implicita ou pra uma `SIMD` *lane*, e faz com que o item da lista original ser atualizado após o fim da região.

9. **`reduction([ reduction-modifier,]reduction-identifier : list)`**: Possibilita a especificação de operações de redução sobre variáveis. Cada *thread* pode ter uma cópia local da variável e ao final é necessário que esses valores parciais sejam acumulados ou atualizados em uma variável.

Cláusulas do construtor `SIMD` vi

10. **`collapse(n)`**: Cláusula utilizada para especificar quantos *loops* estão associados com o construtor. O parâmetro deve ser uma constante inteira positiva. Se a cláusula *collapse* não está presente, somente o *loop* imediatamente seguinte à diretiva é associado ao construtor. Se um ou mais *loops* são associados ao construtor **`simd`** então as iterações de todos os laços são transformadas em um espaço de iteração maior que será executado com `SIMD` instructions.

11. **`order(concurrent)`**: Diz que as iterações dos *loops* associados ao construtor podem ser executadas em qualquer ordem, incluindo em concorrência.

Construtor `#pragma omp declare simd`

Construtor `#pragma omp declare simd` i

- O construtor **`declare simd`** pode ser aplicado a uma função em C. C++ ou Fortran. Ele possibilita a criação de uma ou mais versões que podem ser executadas usando instruções **SIMD**.

Sintaxe

```
#pragma omp declare simd [clause[ [,] clause] ... ] new-line
[#pragma omp declare simd [clause[ [,] clause] ... ] new-line]
[ ... ]
{Definção de uma função ou declaração}
```

No documento de especificação *OpenMP 5.0*: *2.9.3.3 declare simd Directive*

Cláusulas admitidas pelo `declare simd` i

1. **`simdlen(length)`**: Especifica o número preferencial de iterações a serem executadas simultaneamente, a menos que uma cláusula **`if`** esteja presente e seja avaliada como falsa, caso em que o número preferencial de iterações a serem executadas simultaneamente é um. O parâmetro da cláusula simdlen deve ser uma expressão de número inteiro positivo constante.

2. **`linear(linear-list[ : linear-step])`**: Declara um ou mais itens da lista como privados e tendo um relacionamento linear com o espaço de iteração de um *loop* associado ao construtor na qual a cláusula aparece.

Cláusulas admitidas pelo `declare simd` ii

3. **`aligned(argument-list[ : alignment])`**: Declara que o objeto para o qual cada item da lista aponta está alinhado ao número de bytes expresso no parâmetro opcional da cláusula.

4. **`uniform(argument-list)`**: A cláusula **`uniform`** declara que um ou mais argumentos têm um valor invariável para todas as invocações simultâneas da função na execução de um único *loop* `SIMD`.

5. **`inbranch`**: Especifica que a versão `SIMD` da função sempre será chamada de dentro de um condicional de um `SIMD` *loop*.

6. **`notinbranch`**: Esta cláusula especifica que a versão `SIMD` da função nunca será chamada de dentro de um condicional do `SIMD` *loop* .

Cláusulas admitidas pelo `declare simd` iii

- Se nenhuma das duas cláusulas for especificada a versão `SIMD` da função pode ou não ser chamada de dentro de um condicional do `SIMD` *loop* .

Uso combinado do construtor `SIMD`

Uso combinado do construtor SIMD i

- O construtor **simd** pode ser combinado com construtores como o **for** e **taskloop** para que o conjunto de iterações seja dividido entre as *threads* e essas iterações possam ser executadas usando instruções SIMD.

- **Exemplo:** O construtor **simd** combinado com o **for**:

```
int main(int argc, char *argv[]) {
  int i;
  /* Inicialização  dos vetores. */
  init_array();
  #pragma omp parallel for simd schedule(dynamic, 32) num_threads(4)
  for (i = 0; i < N; i++) {
    h_c[i] = h_a[i] * h_b[i];
  }
  /* Resultados. */
  print_array();
```

Uso combinado do construtor SIMD ii

```
    check_result();
    return 0;
  }
```

- **Exemplo:** O construtor **simd** combinado com o **for**.
- O *assembly* gerado para a função de execução do laço com instruções SIMD em seu corpo.

```
.type main._omp_fn.0, @function
main._omp_fn.0:
  /* Código Suprimido. */
  call  GOMP_loop_dynamic_next
  testb %al, %al
  je  .L13
.L17:
  /* Código Suprimido. */
.L15:
  cmpl  %edx, -20(%rbp)
  jge .L14
  movl  -20(%rbp), %eax
  cltq
  movsd h_a(,%rax,8), %xmm1
  movl  -20(%rbp), %eax
  cltq
  jmp .L15
.L14:
  cmpl  $1048576, -20(%rbp)
  je  .L16
.L18:
  /* Código Suprimido. */
  call  GOMP_loop_dynamic_next
  testb %al, %al
  jne .L17
  jmp .L13
.L16:
  /* Código Suprimido. */
  jmp .L18
.L13:
  cmpl  $1048576, %ebx
  je  .L19
```

Uso combinado do construtor SIMD iii

- O construtor **simd** pode também ser combinado com o **taskloop**.
- O exemplo utiliza os construtores **taskloop** e **simd** combinados.
- As iterações do laço serão executadas em paralelo por *tasks* e as iterações que cada *thread* executa podem ser transformadas em instruções SIMD.

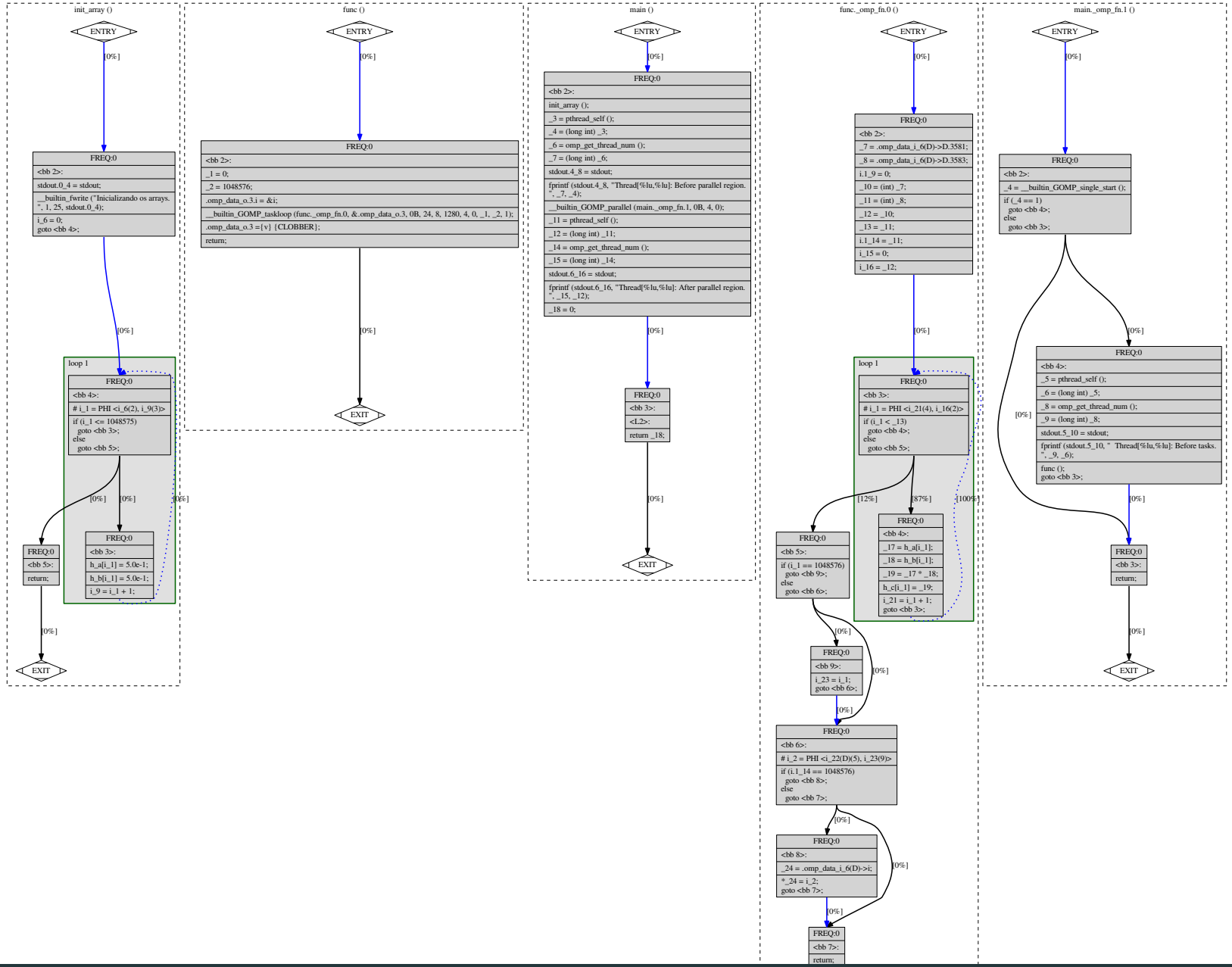
```
void func(){
  int i;

  #pragma omp taskloop simd num_tasks(4)
  for (i = 0; i < N; i++) {
    h_c[i] = h_a[i] * h_b[i];
  }
}
```

Uso combinado do construtor `SIMD` iv

- Toda a estrutura de execução do construtor **`taskloop`** e dos construtores utilizados na região paralela é criada.

Uso combinado do construtor SIMD v



Atividades sobre *simd*

Exercício de Implementação

1. Executar os exemplos para testar a execução com `SIMD`.
2. *Projeto A*: Escolher um *benchmark* (Polybench OpenMP) que utilize originalmente o construtor *for*, alterar ele para utilizar o construtor *taskloop*. Fazer um comparativo utilizando as versões *for*, *taskloop*, *simd* sozinho e combinando *simd* com o *for* e *taskloop*. Execute e compare se há melhoria no desempenho com a utilização de instruções de vetorização (SIMD).

Dúvidas

Perguntas?

Prof. Rogério Aparecido Gonçalves

rogerioag@utfpr.edu.br

Word Cloud


__ Declara
movl rbp
ou
código
n
AVX AVX2
como
vetorização
https
built
pragma omp
são
de
parâmetro
MMX SSE
SSE4 AVX
Support
list
h_a
h_b
cltq movsd
exampleblock
int
SIMD fragile
0 construtor
xmm1
main
uma
em
Código Suprimido
execução
gerado
função
ymm0
Se
loop
número
C int
eax cltq
cláusula
SSE3 SSSE3
rax
ser
da
utilizando
scriptsize
e
SSE2 SSE3
declare simd
eax eax
function
fragile allowframebreaks
normalsize
laço
instruções
XMMWORD PTR
alertblock
na
reduction
code generation
ou mais
b
exemplo
conjunto
SSE4 SSE4
pode ser
ao
OpenMP
iterações
Flag
AVX2 AVX512F
O
linear
rbp eax
um
bit
taskloop
construtores
Especifica
combinado
construtor simd
Intel
SSE SSE2
simd
Support MMX
SSSE3 SSE4
xmm0
h_c
C
da lista
ao construtor
associado
gcc
que
ymm1
para

Referências

Referências i

Flynn, Michael. 2011. “Flynn’s Taxonomy.” In *Encyclopedia of Parallel Computing*, ed. David Padua. Boston, MA: Springer US, 689–97. https://doi.org/10.1007/978-0-387-09766-4_2.

Flynn, Michael J, and Kevin W Rudd. 1996. “Parallel Architectures.” *ACM Computing Surveys (CSUR)* 28(1): 67–70.